SESSION 3

From Python Scripts to AI Backend APIs

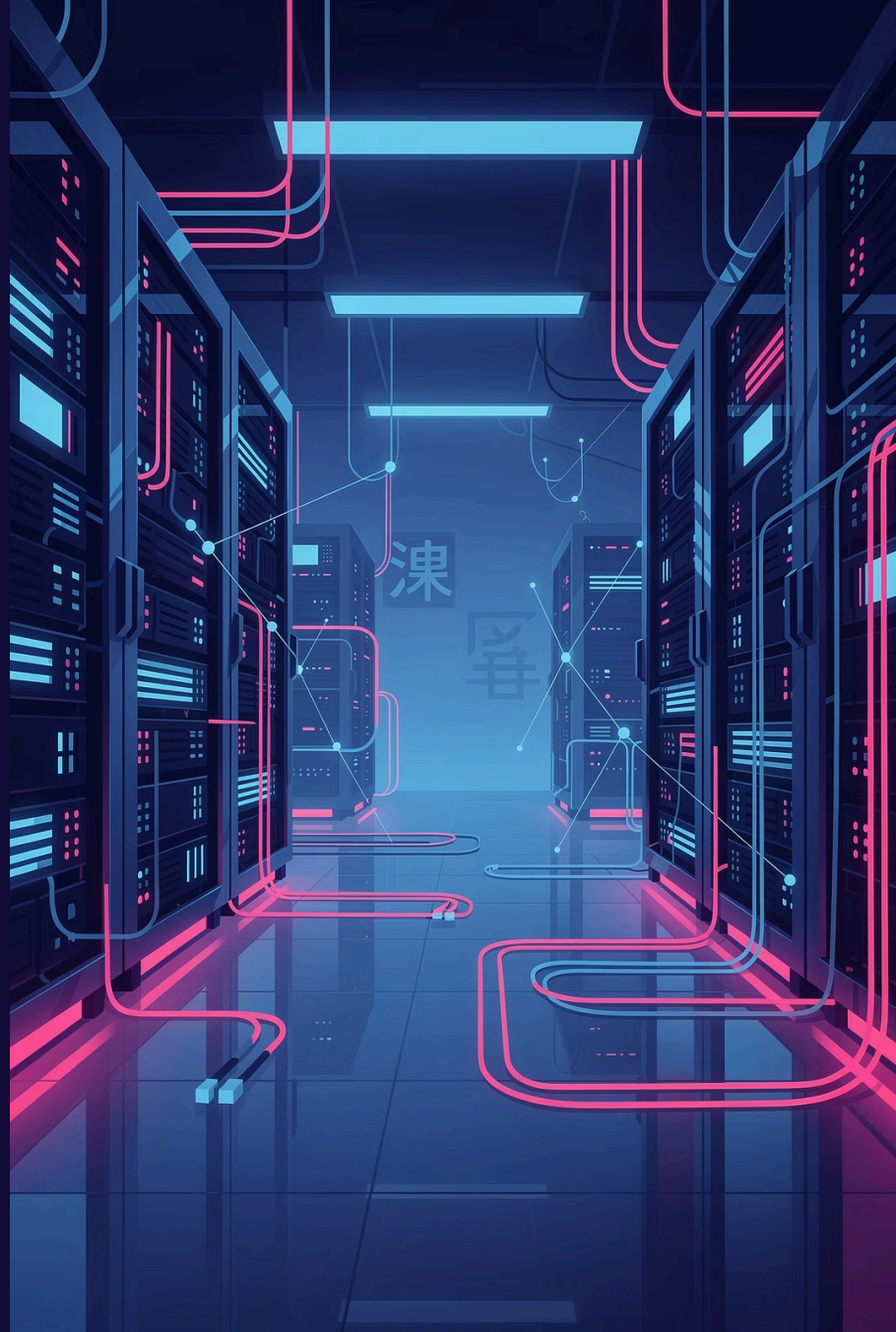
Using FastAPI to expose your AI logic as production-ready backend services

Shrey

AI Solution Engineer

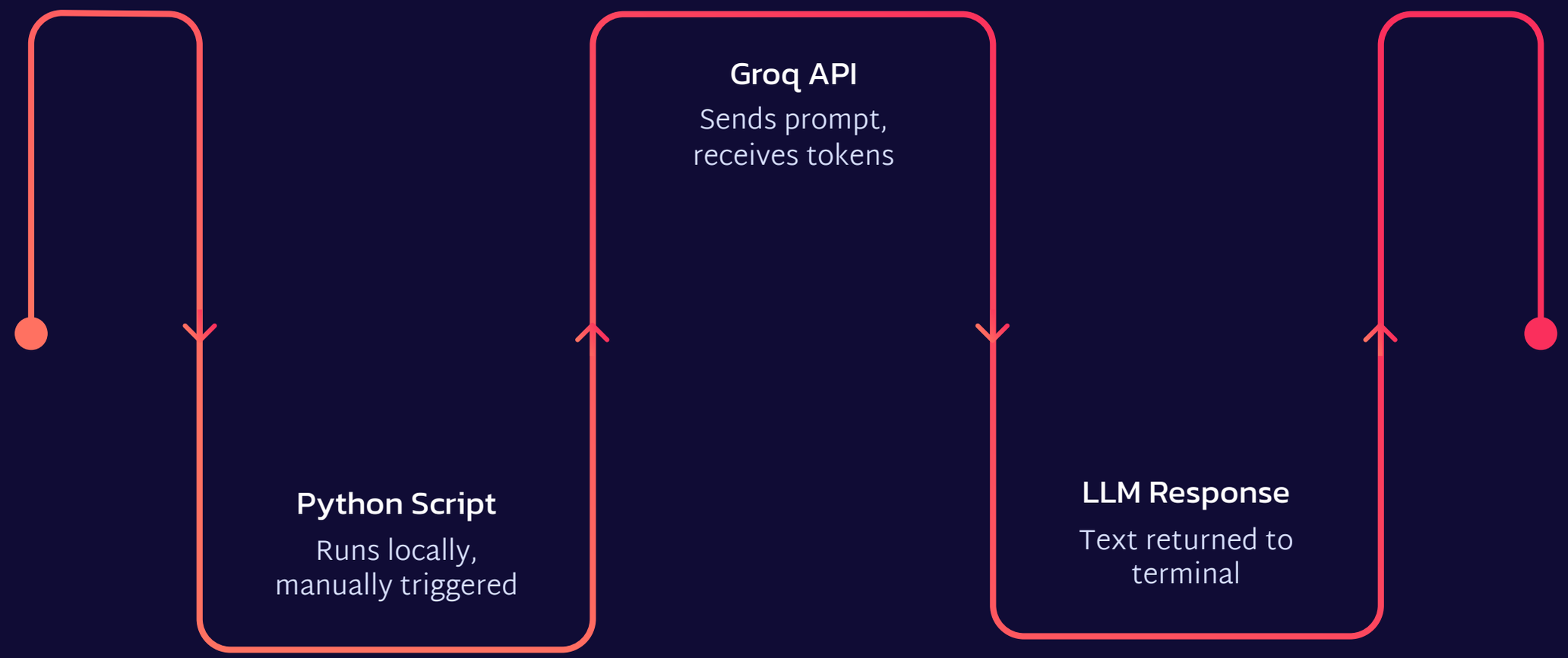
10 March 2026

AI Engineering Series - Session 3



Quick Context: Where We Left Off

In previous session, we built a working Python script that calls the **Groq API** and retrieves a response from a large language model. That was a great first step - but it has a fundamental limitation.



A Python script runs once, on your machine, when you execute it. It cannot be called by a mobile app, a web front-end, or any external system. To make your AI logic **accessible to the world**, you need to wrap it in a backend API.

From Python Script → Backend Service

A Python script and a backend service are fundamentally different things. Understanding this transformation is the first step to building production-ready AI APIs.

Python Script

- Runs once, then exits
- Local execution only
- Single user at a time
- Triggered manually by a developer

Backend Service

- Runs continuously, always listening
- Accessible over the network
- Serves multiple clients simultaneously
- Triggered by incoming HTTP requests

Python Script

FastAPI API Endpoint

Backend Server (Uvicorn)

Multiple Users

Why Backend Frameworks Exist

Without a framework, building even a basic HTTP API in Python requires implementing every layer of the stack by hand. This is error-prone, time-consuming, and completely non-standard.

Routing

Map URLs to handler functions

HTTP Parsing

Decode request bodies and headers

Validation

Verify input types and constraints

Error Handling

Return meaningful HTTP error codes

Serialisation

Convert Python objects to JSON

API Docs

Generate OpenAPI specifications

Frameworks handle all of this - letting you focus on **what your API actually does**, not how HTTP works under the hood.

The Python Backend Framework Landscape

The Python ecosystem has evolved significantly over the past two decades. Each generation of frameworks addressed limitations of its predecessors - culminating in FastAPI's modern, async-first, API-native design.

Django — 2005

Full-stack, batteries-included framework. Built for content-heavy web applications with ORM, templating, and admin panel out of the box.

FastAPI — 2018

Modern, async-first, API-native framework. Automatic validation, documentation, and near-native performance - built for today's AI and microservice workloads.

1 2 3

Flask — 2010

Lightweight micro-framework. Gave developers flexibility to pick their own tools, but left routing, validation, and docs entirely manual.

FRAMEWORK REVIEW

Flask

Flask is a **micro-framework** that gives developers maximum flexibility. It provides just enough structure to handle routing and requests, then gets out of the way. This made it enormously popular for small services and rapid prototyping.

Strengths

- Extremely **lightweight** - minimal overhead
- Highly **flexible** - choose your own libraries
- Large ecosystem of extensions
- Gentle learning curve

Limitations for AI APIs

- **Limited async support** - synchronous by default, blocking under load
- **Manual validation** - no built-in schema enforcement
- **No built-in documentation** - must add Swagger manually
- Not designed with high-concurrency AI workloads in mind

Flask works well for simple APIs, but its synchronous model becomes a bottleneck when waiting on LLM responses.

FRAMEWORK REVIEW

Django

Django is the **full-stack heavyweight** of the Python ecosystem. It ships with virtually everything you need to build a complete web application - but that comprehensiveness comes at a cost when you only need a clean, fast API.

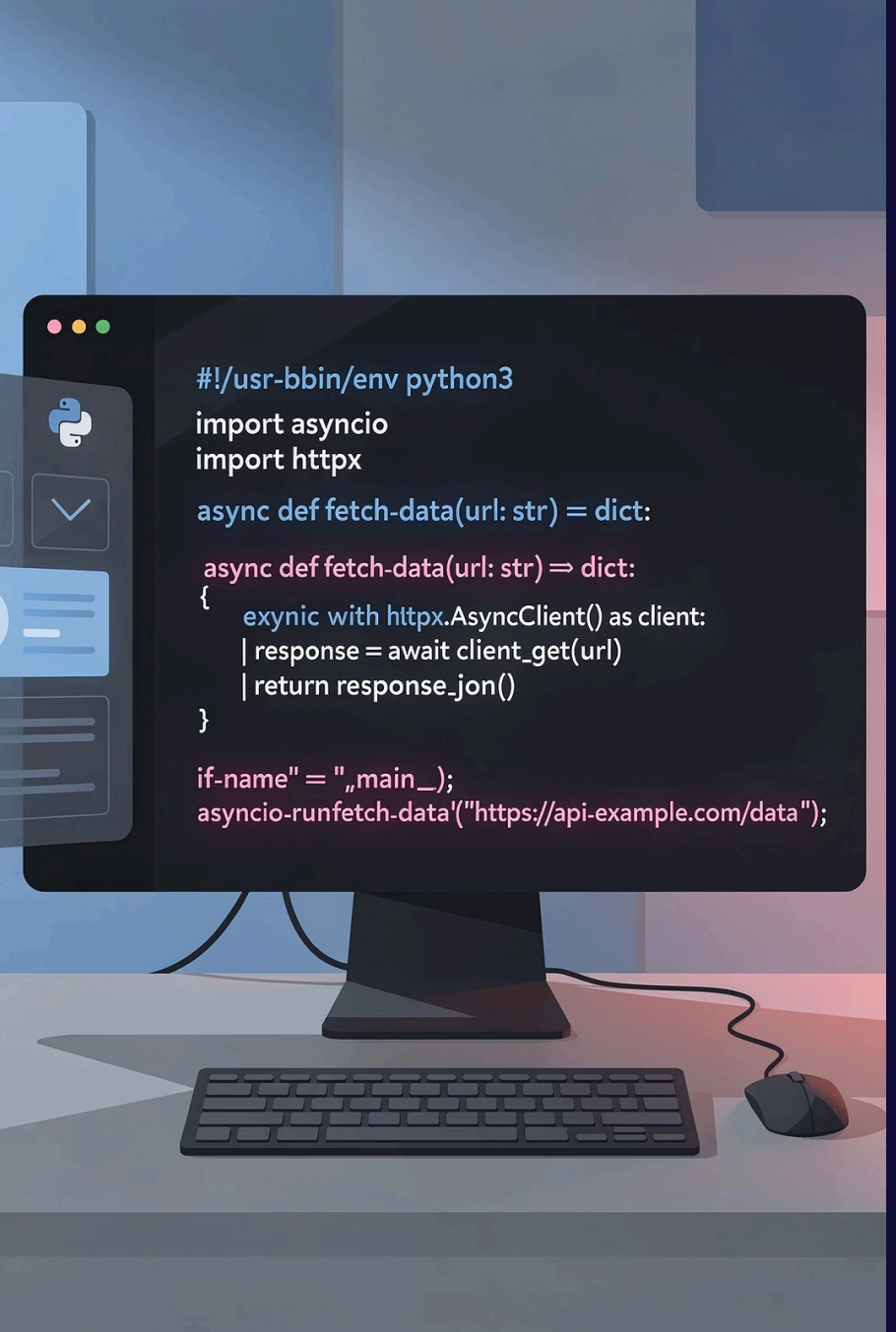
Strengths

- **Full-stack framework** - templating, auth, admin included
- **Built-in ORM** - database models out of the box
- **Admin dashboard** - instant data management UI
- Mature, battle-tested ecosystem

Limitations for AI APIs

- **Heavy framework** - significant startup and runtime overhead
- **Monolithic design** -- difficult to decompose into microservices
- **Not API-first** - Django REST Framework needed as an add-on
- Async support added late and remains incomplete

Django REST Framework is powerful but adds complexity. For pure API workloads, Django carries significant unnecessary overhead.



INTRODUCING FASTAPI

FastAPI: Built for the Modern API Era

FastAPI is a **modern, high-performance Python framework** designed from the ground up for building APIs. Created by **Sebastián Ramírez in 2018**, it leverages Python's type hints and async capabilities to deliver developer productivity without sacrificing performance.

AI APIs

Wrap LLM calls in production-grade endpoints with full validation

ML Inference

Serve model predictions at scale with async concurrency

Microservices

Build lightweight, independently deployable service components

Why FastAPI Became the Standard for AI APIs

FastAPI's rapid adoption in the AI and ML engineering community is no accident. It solves the exact pain points that matter most when building systems that interact with large language models and inference services.

Extreme Performance

Benchmarks show FastAPI on par with **Node.js and Go** - the fastest Python framework available.

Async Architecture

Built on **ASGI and asyncio**, enabling non-blocking I/O for concurrent LLM requests.

Automatic Docs

Swagger UI and ReDoc generated automatically from your code - zero extra effort.

Pydantic Validation

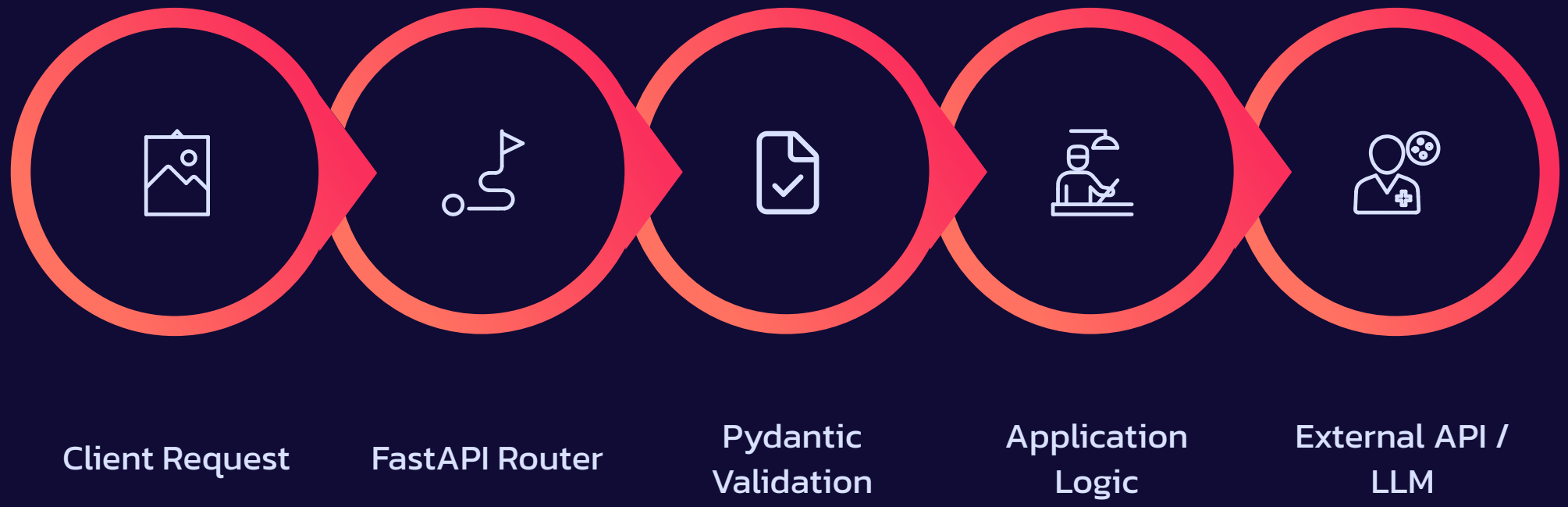
Type-safe request and response models enforced automatically - no manual checks needed.

Developer Productivity

Python type hints provide **full IDE autocompletion** and dramatically reduce bugs.

FastAPI Request Lifecycle

Every request that enters your FastAPI application passes through a well-defined lifecycle. Understanding this pipeline helps you know *exactly where* to add validation, logging, error handling, and business logic.



If validation fails at step 3, FastAPI automatically returns a **422 Unprocessable Entity** response with a detailed error message — no custom error handling required.

Typical FastAPI Project Structure

A well-structured FastAPI project separates concerns cleanly. This layout scales from a single-file prototype to a production-grade AI service.

```
app/
 ├── main.py        # App entry point, mounts routers
 ├── routers/
 │   ├── chat.py     # /chat endpoints
 │   └── health.py   # /health check
 ├── services/
 │   ├── llm.py      # Groq/OpenAI API calls
 │   └── retrieval.py # Vector store queries
 └── models/
     ├── request.py  # Pydantic input schemas
     └── response.py # Pydantic output schemas
```

routers/

Define API endpoints and HTTP methods. Each router groups related routes (e.g., all /chat routes in chat.py).

services/

Contain business logic and external API calls. Routers call services — keeping endpoints thin and logic testable.

models/

Pydantic schemas for request validation and response serialization. Single source of truth for data shapes.

This separation makes your codebase testable, maintainable, and easy to extend as your AI system grows.

AI Backend Architecture: Where FastAPI Fits

In production AI systems, FastAPI sits at the center — bridging client applications and external AI providers through a clean, layered architecture.

User / Client

Web App, Mobile App, or CLI tool sending HTTP requests

API Endpoint (POST /ai/chat)

FastAPI route that receives and validates the incoming request via Pydantic

FastAPI Backend Service

Orchestrates the request — authentication, rate limiting, logging, routing to services

Service Layer

Business logic: prompt construction, context management, response formatting

External AI Provider (Groq LLM API)

LLM inference — model processes the prompt and streams or returns a completion

Response → Client

Serialized JSON response returned with HTTP status code

FastAPI is the backbone of this architecture — it handles everything between the client and the AI provider. In the next section, we'll build this layer by layer.

FastAPI vs Flask vs Django

Choosing the right framework depends on your use case. For AI APIs and ML inference services, the comparison is clear - FastAPI wins on nearly every dimension that matters.

Framework	API-First	Async Support	Auto Docs	Validation	Performance	Best Use Case
FastAPI	Yes	Native	Built-in	Pydantic	Very High	AI APIs, ML inference, microservices
Flask	Partial	Limited	Manual	Manual	Moderate	Simple REST APIs, prototyping
Django	No	Bolted-on	Add-on	DRF only	Lower	Full-stack web apps, admin tools

WSGI vs ASGI: Server Communication Models

Understanding server interface models is essential to understanding why FastAPI excels under concurrent AI workloads. The core distinction lies in how they handle I/O: **blocking** (WSGI) versus **non-blocking** (ASGI) operations.

WSGI (Synchronous)

- Handles one request at a time per worker
- Blocks the execution thread while waiting for I/O
- Used by Flask and Django (default)
- Suboptimal for high-latency, I/O-heavy AI workloads

ASGI (Asynchronous)

- Handles many requests concurrently per worker
- Non-blocking: yields thread control while awaiting I/O
- Used by FastAPI + Uvicorn
- Ideal for LLM inference, streaming, and real-time APIs

Execution Flow Comparison

WSGI Flow

Request → Handle → Wait (Blocked) → Respond

ASGI Flow

Request → Handle → Await (Free) → Handle Others → Respond

FastAPI uses ASGI - enabling high-throughput, concurrent AI APIs.

Why Async Concurrency Matters for AI APIs

AI APIs are fundamentally I/O-bound. Every call to an LLM, vector store, or external service introduces latency - often measured in seconds. How your server handles that wait time determines your system's throughput.

LLM Inference

Groq, OpenAI, etc. — 0.5s to 10s per call

Vector Store Queries

Pinecone, Chroma — network round-trip

Database Operations

PostgreSQL, Redis — read/write latency

External API Calls

Third-party integration services

Execution Model Comparison

Synchronous (Blocking)

Request A → [LLM Wait 3s] → Respond

Request B must wait the entire time.

Async (Non-Blocking)

Request A → [LLM Wait 3s — server free] → Respond

Request B is handled during Request A's wait.

With async/await, FastAPI processes other requests while waiting for LLM responses - dramatically improving throughput under real-world AI workloads.

Running a FastAPI Server

FastAPI applications are served by **Uvicorn**, a lightning-fast ASGI server. Starting your development server is a single command.

```
uvicorn app.main:app --reload
```

`uvicorn`

The **ASGI server** that handles HTTP connections, event loops, and request dispatching. Think of it as the runtime that hosts your FastAPI app.

`app.main`

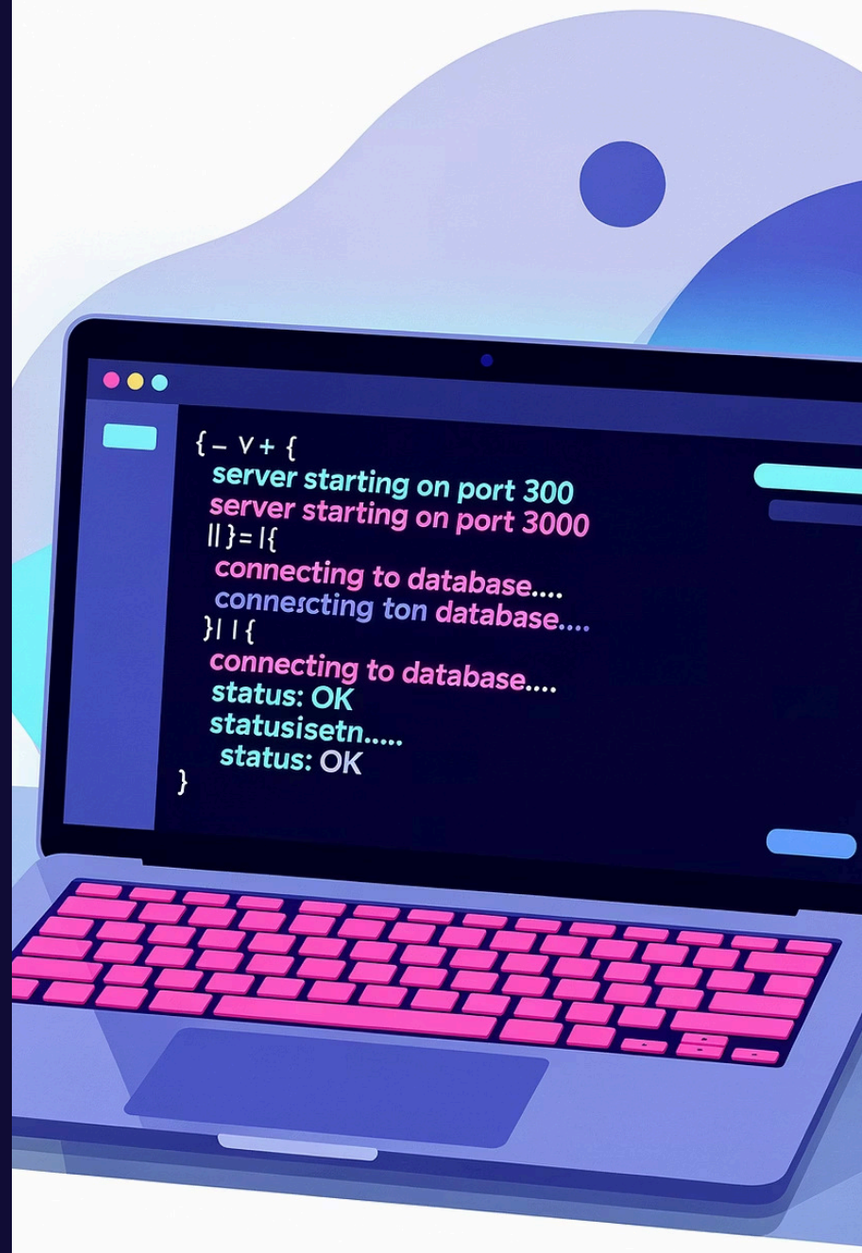
The **Python module path** to your application file - equivalent to `app/main.py` in your project directory.

`app`

The **FastAPI instance** variable inside `main.py` — typically `app = FastAPI()`.

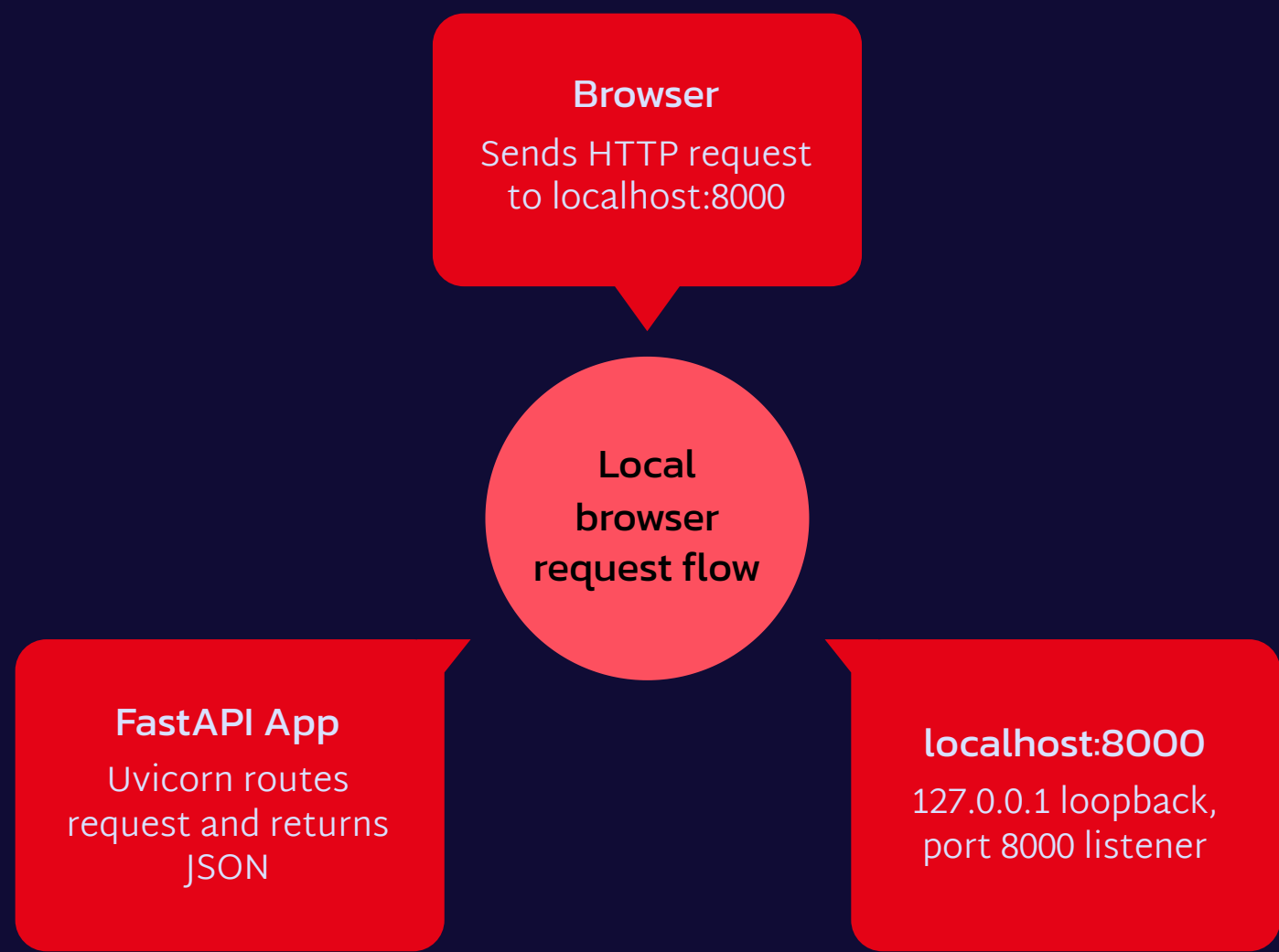
`--reload`

Enables **hot reloading** - the server automatically restarts when you save a file. Development-only flag; never use in production.



The Local Development Server

When Uvicorn starts, your FastAPI application is accessible at `http://localhost:8000`. Understanding what this address means is foundational for local development and debugging.



`localhost`

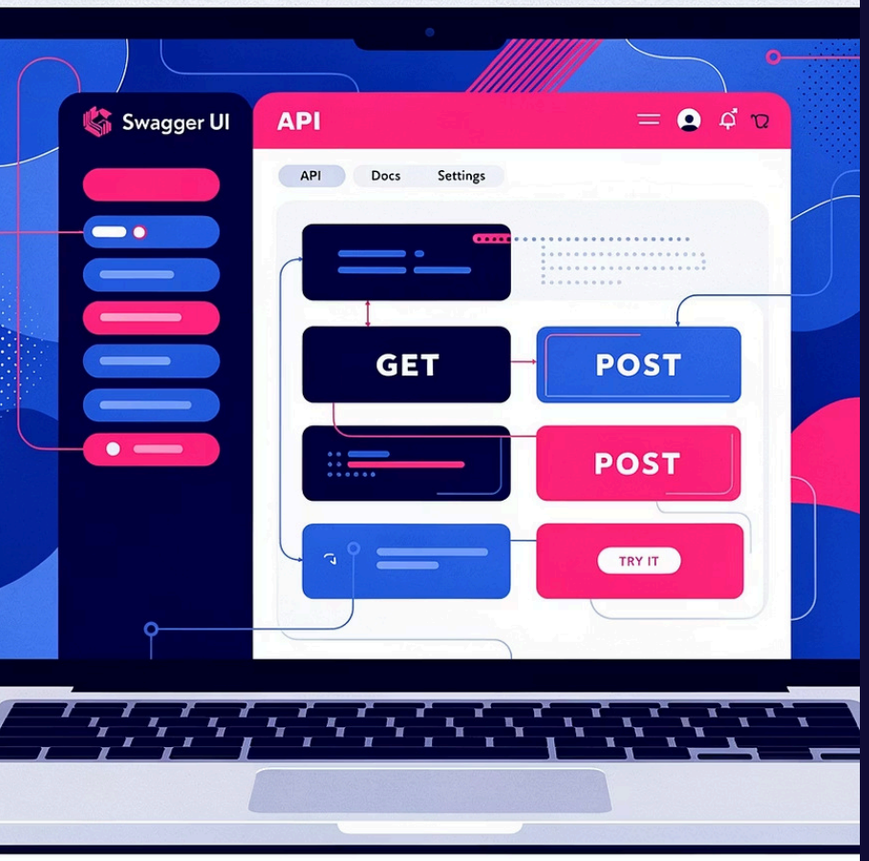
Resolves to `127.0.0.1` - your own machine's loopback address. Traffic never leaves your computer.

Port `8000`

The network port Uvicorn listens on. Change with `--port 8080` if needed.

Dev Server

Uvicorn with `--reload` is for development only. Production uses Gunicorn + Uvicorn workers.



Automatic API Documentation

One of FastAPI's most celebrated features is **zero-effort API documentation**. The moment you define your endpoints and Pydantic models, FastAPI generates a fully interactive documentation interface — no extra code required.

Swagger UI

`http://localhost:8000/docs`

- **Interactive testing** - send real requests directly from the browser
- Full **request and response schemas** displayed
- Instant **debugging** without needing Postman or curl
- Auto-updates as you change your code

ReDoc

`http://localhost:8000/redoc`

- Clean, **read-optimised** documentation layout
- Great for sharing API specs with **frontend teams**
- Both powered by the same **OpenAPI 3.0** schema FastAPI generates automatically

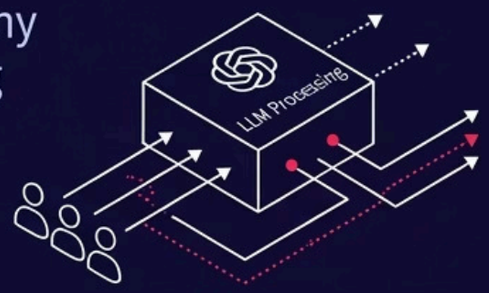
Both UIs are generated from a single OpenAPI JSON schema at `/openapi.json` — you can also feed this into any OpenAPI-compatible tool.

Why FastAPI Is the Right Choice for AI Systems

Every architectural decision in FastAPI aligns with the demands of AI infrastructure. This isn't coincidence — FastAPI was built precisely for the era of networked, I/O-heavy, intelligent services.

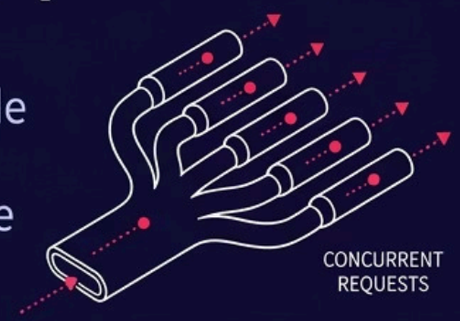
Async for Long Model Calls

Server handles many users while waiting on LLM token generation.



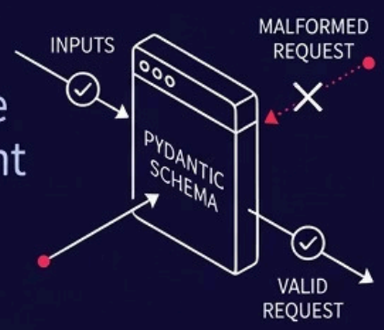
High Throughput

ASGI and Uvicorn deliver near Go/Node performance for concurrent inference requests.



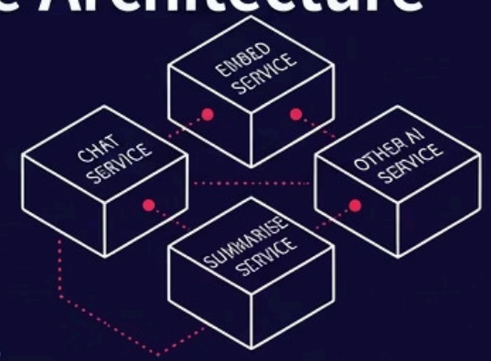
Automatic Validation

Pydantic models enforce prompt schemas, prevent malformed requests, and document the contract automatically.



Microservice Architecture

Lightweight and composable, each AI capability becomes its own deployable service.



As AI systems grow in complexity — chaining models, retrieving from vector stores, calling multiple APIs - FastAPI's async-first, validation-native, documentation-automatic design becomes increasingly valuable.

UP NEXT

What's Coming next?

Now that you understand *why* FastAPI is the right foundation for AI APIs, it's time to get your hands on the actual code. Now, we move from theory to practice.

Explore the Project Structure

Walk through the FastAPI project layout — routers, schemas, services, and configuration - and understand how each layer maps to the architecture we've discussed today.

Examine the Codebase

Read through the starter code, understand the patterns, and see how Pydantic models, async route handlers, and dependency injection work together in practice.

Build Our First AI API Endpoint

Write a real `/chat` endpoint that accepts a user message, calls the Groq API asynchronously, and returns a structured LLM response - live, in session.